



EESC-USP Formula SAE: Drivetrain Analysis

Manual de referência, uso e manutenção — PlotterPowertrain v3

Arquivo analisado: PlotterPowertrain_v3.py

Total: 3582 linhas

Versão documental gerada em 04/10/2026

SHA-256 do código: c418ab51cbfe9cca...

Escopo: Este documento descreve o código como ele existe no arquivo analisado. As explicações de equações, fluxo e limitações refletem a implementação observada; o documento não pressupõe que o modelo físico seja uma representação completa do veículo.

1. Visão geral e objetivo

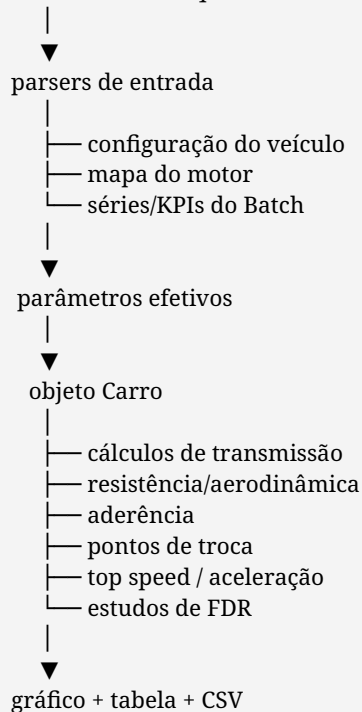
O PlotterPowertrain é uma aplicação Python para analisar o powertrain de um carro de Fórmula SAE com foco em relações de transmissão finais (FDR), curvas de motor, força trativa, potência na roda, pontos de troca, desempenho longitudinal e comparação com resultados de Batch Run do OptimumLap.

A arquitetura atual separa os dados de entrada em três fontes independentes e concentra os cálculos físicos na classe Carro. As interfaces de terminal e gráfica funcionam como camadas de entrada/apresentação/exportação.

Componente	Responsabilidade
carro.csv	Configuração principal do veículo: relações, raio, massa, coeficientes e parâmetros físicos opcionais.
CSV RPM × Torque	Mapa discreto de torque do motor em função de RPM.
Batch Run OptimumLap	Resultados de múltiplos runs e parâmetros/KPIs usados na validação.
Classe Carro	Modelo matemático: transmissão, resistências, aderência, top speed, aceleração e estudos de FDR.
InterfacePlotterPowertrain	Interface de terminal, solicitação de parâmetros, armazenamento e exportação.
PlotterPowertrainGUI	Interface gráfica Tkinter + Matplotlib, tabelas, gráficos e exportação.

2. Arquitetura e fluxo de dados

carro.csv + RPM×Torque + Batch Run



A GUI instancia InterfacePlotterPowertrain e reutiliza sua estrutura de resultados. Os arquivos permanecem independentes: é possível trocar apenas o mapa do motor ou apenas o Batch Run sem necessariamente substituir a configuração do carro.

3. Arquivos de entrada

3.1 carro.csv

O formato preferencial é uma tabela de duas ou mais colunas separada por ponto e vírgula, na qual a primeira coluna identifica o parâmetro e as seguintes contêm o valor. A leitura é feita pelo nome do parâmetro, não pela posição da linha.

```
nome;Teste
relacoes_marcha;GearRatios.csv
relacao_primaria;2.6667
relacao_secundaria;2.9090
raio_roda;0.194
peso;240
Cat_pneu;2.204
```

Parâmetros físicos opcionais reconhecidos pelo parser incluem eficiência do drivetrain, Cd, Cl, área frontal, densidade do ar, resistência ao rolamento, limite de RPM, atrito longitudinal/lateral, distribuição de carga traseira, distribuição de downforce traseiro, altura do CG e entre-eixos.

Parâmetro	Unidade	Papel no modelo
relacao_primaria	-	Relação primária usada no cálculo do FDR.
relacao_secundaria	-	Relação secundária; FDR = primária × secundária.
raio_roda	m	Converte torque/rotação em força/velocidade.
peso	kg	Massa utilizada em resistência, carga normal e aceleração.
Cat_pneu	-	Coefficiente histórico/base de aderência.
drivetrain_efficiency	0–1 ou %	Eficiência entre motor e roda.
drag_coefficient	-	Cd da resistência aerodinâmica.
downforce_coefficient	-	Cl usado no downforce.
frontal_area	m ²	Área frontal A.
air_density	kg/m ³	Densidade do ar ρ .
tire_rolling_drag	-	Coefficiente de resistência ao rolamento Crr.
rev_limit	rpm	Limite declarado de rotação.
longitudinal_friction	-	Coefficiente longitudinal μ_x .
rear_weight_distribution	fração	Parcela estática de massa no eixo traseiro.
rear_downforce_distribution	fração	Parcela do downforce atribuída ao eixo traseiro.
cg_height	m	Altura do centro de gravidade, usada na transferência longitudinal.
wheelbase	m	Entre-eixos, usado na transferência longitudinal.

Caminhos de arquivo: A função getCarroConfig aceita caminho relativo e procura-o relativo à pasta do carro.csv. Caminhos absolutos Windows também são reconhecidos; se não existirem na máquina atual, o programa tenta localizar pelo nome do arquivo na pasta do carro.csv.

3.2 Curva RPM × Torque

A função `getRPMxTorquePoints` aceita arquivos delimitados por TAB, ponto e vírgula ou vírgula, desde que a detecção do delimitador seja coerente com a amostra. A curva é devolvida como pares [RPM, torque], ordenados de forma crescente em RPM.

```
3500    19,10
3750    20,00
4000    20,50
...
```

Ponto decimal e vírgula decimal são aceitos. Linhas vazias são ignoradas. Um cabeçalho textual é tolerado na primeira linha.

3.3 Batch Run do OptimumLap

O parser `getBatchRun` lê as seções e guarda as séries numéricas em um dicionário indexado pelo primeiro campo da linha. Isso elimina a dependência de posições como `batch[37]`.

```
Final Drive Ratio [-];6.5;6.65;6.8;...
Lap time [s];45.091;45.084;45.075;...
Gear Shifts [-];60;62;62;...
```

Regra de manutenção: Ao criar um novo estudo de Batch Run, prefira sempre `_buscar_linha_por_rotulo(...)` e aliases, em vez de armazenar ou assumir um índice fixo da série.

4. Bibliotecas e funções utilitárias

Função / módulo	Finalidade
<code>csv</code>	Leitura e escrita de CSVs.
<code>numpy (np)</code>	Malhas numéricas, <code>arange/linspace</code> e apoio à GUI.
<code>matplotlib.pyplot (plt)</code>	Criação dos gráficos.
<code>MultipleLocator</code>	Controle de ticks dos eixos.
<code>time</code>	Medição de tempo de execução.
<code>math</code>	Constantes e funções matemáticas, especialmente π .
<code>Pathlib.Path</code>	Manipulação de caminhos de arquivo.
<code>re</code>	Detecção de caminhos Windows.
<code>Tkinter/ttk</code>	Interface gráfica.

Importações não essenciais observadas: `matplotlib.path` (`pe`) e `ctypes` aparecem importados no arquivo, mas não são utilizados pelo fluxo atual identificado. `plotterPrint` também existe como função auxiliar, mas não é uma dependência central do fluxo.

4.1 `setMatplotlibParameters()`

Configura globalmente o estilo do Matplotlib: fontes, tamanhos, cores, grids, espessuras, legenda, dimensão padrão da figura e DPI. A variável global `rc_isdef` evita reaplicações desnecessárias e, na versão analisada, recebe explicitamente global dentro da função.

4.2 `interp_1d(x_new, x, y)`

Implementa manualmente uma interpolação linear 1D. Para cada ponto novo x_i :

```

 $x_i \leq x_0 \rightarrow \text{retorna } y_0$ 
 $x_i \geq x_n \rightarrow \text{retorna } y_n$ 
 $x_k \leq x_i \leq x_{k+1} \rightarrow$ 
    slope =  $(y_{k+1} - y_k) / (x_{k+1} - x_k)$ 
     $y(x_i) = y_k + \text{slope} \cdot (x_i - x_k)$ 

```

A função pressupõe que x está ordenado e não possui dois pontos consecutivos com a mesma abscissa. Fora do domínio, ela mantém o valor extremo (extrapolação constante).

4.3 linspace_pure(start, stop, num)

Gera manualmente uma lista de num pontos igualmente espaçados entre start e stop, incluindo os extremos quando num > 1.

5. Parsers de dados

_detectar_delimitador_csv() — linhas 233–248

Inspeciona os primeiros 4096 caracteres e prioriza TAB, depois ;, depois vírgula.

getRPMxTorquePoints() — linhas 249–289

Converte linhas em [RPM, torque], valida pelo menos dois pontos e ordena por RPM.

getGearRatios() — linhas 290–321

Extraí o segundo campo das linhas de relação de marcha e ordena as relações em ordem decrescente.

_normalizar_rotulo() — linhas 322–329

Normaliza caixa, espaços e acentos para comparações robustas.

_primeiro_valor_numerico() — linhas 330–339

Tenta converter uma célula em float, retornando None quando não for numérica.

_extrair_series_numericas() — linhas 340–351

Percorre campos depois do rótulo e coleta todos os valores numéricos.

_buscar_linha_por_rotulo() — linhas 352–372

Procura uma série do Batch pelo rótulo exato e, em segundo nível, pelo rótulo normalizado/alias.

getBatchVehicleParameters() — linhas 372–409

Extraí parâmetros físicos do Batch, convertendo percentuais como 96% em 0.96.

getBatchRun() — linhas 409–474

Lê metadados, seções e séries numéricas do Batch Run.

getCarroCSV() — linhas 550–567

Mantém compatibilidade antiga: devolve apenas a segunda coluna.

_caminho_eh_absoluto() — linhas 567–580

Reconhece caminhos absolutos locais e caminhos Windows no texto.

getCarroConfig() — linhas 580–886

Converte o carro.csv em um dicionário padronizado, aplica aliases, fallback legado e validações.

6. Classe Carro — modelo físico

A classe Carro é o núcleo do programa. Ela recebe o mapa do motor, as relações de marcha, as relações primária/secundária, raio de roda, massa, aderência e parâmetros opcionais. A partir dessas entradas, constrói curvas de potência/torque/força e estudos de desempenho.

6.1 Inicialização (__init__)

O construtor padroniza os dados, ordena o mapa de motor por RPM e calcula potência do motor ponto a ponto.

```
P_motor[kW] = RPM · Torque[N·m] / 9549.2966
P_motor[HP] = P_motor[kW] · 1.34102
```

As relações de marcha são ordenadas em ordem decrescente para que o índice 1 represente a maior relação, interpretada como 1ª marcha. A relação final é definida por:

```
FDR = relação_primária · relação_secundária
```

A eficiência do drivetrain é convertida para fração caso tenha sido fornecida em porcentagem. O limite de RPM efetivo é:

```
RevLimit_efetivo = min(RevLimit_declarado, RPM_máximo_do_mapa)
```

Consequência prática: Se o Batch disser Rev Limit = 10.931 rpm mas a curva fornecida terminar em 10.200 rpm, o modelo não inventa torque acima de 10.200 rpm. Ele pode, porém, avisar que o limitador declarado excede a cobertura do mapa.

6.2 _gerar_mapa_operacional()

Recorta o mapa até o limite de RPM efetivamente simulável. Quando o limite está entre dois pontos do mapa, interpola o torque no limite e adiciona esse ponto. A potência operacional é calculada em seguida.

6.3 propriedades do mapa e potência

Método	Entrada	Saída / finalidade
mapa_cobre_limite_rev	nenhuma	Booleano indicando se o Rev Limit declarado excede o mapa.
potencia_motor_hp(rpm)	rpm	Potência do motor interpolada no mapa operacional [HP].
potencia_roda_hp_em_rpm(rpm)	rpm	Potência na roda = potência do motor × eficiência.

6.4 Curvas por marcha

curvas_potencia(fdr)

Para cada marcha, o código usa a mesma curva de potência na roda, deslocada para a velocidade correspondente à relação total.

```

i_total = i_marcha · FDR
v[km/h] = (RPM/60) · (2πr / i_total) · 3.6
P_roda[HP] = P_motor[HP] · η

```

curvas_torque(fdr)

Calcula torque efetivo na roda após a eficiência do drivetrain:

```
T_roda = T_motor · i_total · η
```

curvas_marcha(fdr)

Calcula a força trativa disponível por marcha:

```
F_marcha = (T_motor · i_total · η) / r
```

Essas três funções são a base para praticamente todos os estudos posteriores.

7. Resistências, aerodinâmica e aderência

7.1 forca_resistencia(v)

A resistência total é a soma de arrasto aerodinâmico e resistência de rolamento:

```

F_drag = 0.5 · ρ · Cd · A · v²
F_roll = Crr · m · g
F_res = F_drag + F_roll

```

A velocidade é convertida de km/h para m/s antes da aplicação das equações aerodinâmicas.

7.2 downforce_total(v)

```
F_down = 0.5 · ρ · Cl · A · v²
```

O código considera downforce positivo como aumento da carga normal nos pneus.

7.3 _normal_traseira(v, aceleracao)

Para um carro com eixo motriz traseiro, o modelo estima a carga normal no eixo traseiro como a soma da parcela estática de peso, parcela traseira do downforce e, quando CG/entre-eixos existem, uma aproximação da transferência longitudinal durante aceleração:

```

N_static = m · g · f_rear
N_aero = F_down · f_down_rear
N_transfer = m · a · h_CG/L
N_rear = N_static + N_aero + N_transfer

```

7.4 _limite_atrito(v, aceleracao)

O limite de força no eixo motriz é modelado por:

```
F_tire,max = μ_longitudinal · N_rear
```

Limitação do modelo: A implementação trata distribuição estática e distribuição de downforce como parâmetros globais. Não existe um modelo detalhado de suspensão, pitch, centro de pressão variável, cambagem, temperatura de pneu ou mapa de μ em função da carga/velocidade.

8. Tração disponível e desempenho longitudinal

8.1 `forca_trativa_disponivel(v, fdr, marcha, considerar_atrito)`

Seleciona a maior força disponível entre as marchas que conseguem atingir a velocidade, ou somente a marcha solicitada. Se o limitador de aderência estiver ativo, a força do powertrain é limitada pelo pneu. Quando há parâmetros de CG e entre-eixos, ocorre uma pequena iteração para incorporar a dependência da carga normal com a aceleração estimada.

```
F_powertrain = max(F_marcha(v))
F_trativa = min(F_powertrain, F_tire, max) [com aderência]
F_trativa = F_powertrain [sem aderência]
```

8.2 `forca_liquida(v)`

```
F_liquida = F_trativa - F_res
```

8.3 `velocidade_maxima_marcha(marcha, fdr)`

É a velocidade cinemática da marcha no limite de RPM efetivo:

```
V_max,marcha = (RPM_limite/60) * (2πr / (i_marcha·FDR)) * 3.6
```

Este valor é um limite geométrico/cinemático da marcha, não o top speed físico do veículo.

8.4 `calcular_top_speed()`

Agora o top speed físico procura onde a força líquida deixa de ser positiva. O código varre uma malha de velocidades até o máximo cinemático das marchas e procura a transição $F_{liquida} \geq 0$ para $F_{liquida} < 0$. A raiz é interpolada linearmente entre os dois pontos.

```
F_liquida(V*) = 0
≈ equilíbrio: F_trativa(V*) = F_drag(V*) + F_roll
```

Interpretação: O resultado depende do mapa de motor disponível, das perdas e dos parâmetros aerodinâmicos. Portanto, não deve ser confundido com simplesmente converter o Rev Limit em km/h.

8.5 `calcular_aceleracao_maxima()`

Varre a faixa de velocidades até o top speed e calcula a maior aceleração longitudinal disponível:

```
a(V) = F_liquida(V) / m
```

8.6 `calcular_pontos_troca()`

Compara curvas de força de marchas consecutivas. Em vez de exigir igualdade exata de uma coordenada discreta, calcula a diferença $F_i(V) - F_j(V)$, procura mudança de sinal e interpola a velocidade de cruzamento.

```
ΔF(V) = F_marcha_atual(V) - F_marcha_seguinte(V)
se ΔF_k > 0 e ΔF_{k+1} ≤ 0:
  V_shift = V_k + [ΔF_k / (ΔF_k - ΔF_{k+1})] · (V_{k+1} - V_k)
```

A velocidade encontrada é convertida novamente em RPM na marcha de origem e na marcha de destino.

9. Simulação longitudinal

9.1 `simular_aceleracao_fdr()`

Realiza uma integração temporal simples da velocidade. O modelo usa passo de tempo dt . A cada passo:

```
F_trativa = min(F_powertrain, F_pneu)    (se aderência ativa)
F_liquida = F_trativa - F_res
a = F_liquida / m
v_{novo} = v + a·dt
```

A distância é atualizada pela média trapezoidal das velocidades no intervalo:

$$\Delta s = 0.5 \cdot (v + v_{\text{novo}}) \cdot dt$$

Quando o veículo atinge a velocidade alvo dentro de um passo, o tempo do alvo é interpolado linearmente. Trocas de marcha são tratadas como velocidade constante durante `shift_time`; o cronômetro é incrementado nesse período.

Largada: O mapa do motor fornecido pode começar acima de 0 rpm. Como não existe um modelo de embreagem/largada, a implementação usa o primeiro ponto do mapa para iniciar a simulação até a velocidade entrar no domínio da curva. Isso é uma aproximação importante para o trecho 0→alvo.

9.2 `avaliar_fdrs()`

Executa a simulação para todos os FDRs fornecidos e organiza, para cada um, tempo até a velocidade alvo, top speed, aceleração máxima e número de trocas. É a principal rotina analítica para comparar FDRs com métricas diretamente relacionadas ao desempenho longitudinal.

10. Estudos complementares e legado

`drawRPMxTorquexPower()`

Gráfico de torque e potência do motor, com eixos y independentes. Marca máximos e o Rev Limit efetivo/declarado.

`gerar_pot_vroda()`

Gera potência na roda por marcha usando `curvas_potencia` e marca os pontos de troca.

`gerar_troda_vroda()`

Gera torque na roda por marcha e marca os pontos de troca.

`gerar_froda_vroda()`

Gera força trativa por marcha e, opcionalmente, o limite de aderência.

`gerar_curva_unica_froda_vroda_v1()`

Constrói uma envoltória escolhendo a maior força disponível em cada velocidade; versão de referência/legado.

gerar_curva_unica_froda_vroda_v2()

Versão atual da envoltória, com FDR explícito, eficiência e opção de aderência.

10.1 Envoltória de força

$$F_{\text{envoltória}}(V) = \max\{F_1(V), F_2(V), \dots, F_n(V)\}$$

Isso representa a maior força disponível entre as marchas, não uma integração temporal completa. A simulação longitudinal é quem tenta representar o histórico de velocidade e trocas.

10.2 calcular_perda_area_potencia()

É uma métrica secundária mantida no código. A força ideal é derivada da potência máxima disponível na roda e a força real é a envoltória. A potência é convertida para watts e dividida pela velocidade em m/s:

$$F_{\text{ideal}} = P_{\text{max,roda}} / v$$

As áreas são integradas pela regra do trapézio sobre a velocidade, e a razão final é:

$$\text{Área}_{\text{ideal}} = \int F_{\text{ideal}}(V) dV$$

$$\text{Área}_{\text{real}} = \int F_{\text{real}}(V) dV$$

$$\text{Perda}_{\text{relativa}} = (\text{Área}_{\text{ideal}} - \text{Área}_{\text{real}}) / \text{Área}_{\text{ideal}}$$

Uso recomendado: O próprio código classifica essa métrica como secundária. Ela pode ser útil para visualizar aproveitamento de força frente a uma curva de potência constante, mas não deve, isoladamente, definir o FDR ótimo.

10.3 gerar_pot_vroda_unificada() e comparar_pot_vroda_fdrs()

Essas rotinas calculam, para cada velocidade, a maior potência na roda disponível entre as marchas e permitem comparar vários FDRs no mesmo gráfico. A mudança do FDR altera principalmente o mapeamento RPM→velocidade; a potência do motor em um mesmo RPM permanece limitada pelo mapa.

11. Integração com Batch Run do OptimumLap

11.1 getBatchVehicleParameters()

Extraí do Batch parâmetros físicos que podem completar o carro.csv. Por exemplo, 96% de eficiência vira 0.96. A lógica de prioridade é: carro.csv primeiro; Batch somente preenche chaves ausentes.

11.2 Estudos de laptime

plotBatchLaptimeFDR mostra o laptime em função do FDR. plotBatchLaptimesFDRShift adiciona uma penalização simplificada de troca:

$$t_{\text{lap,ajustado}} = t_{\text{lap}} + N_{\text{trocas}} \cdot t_{\text{shift}}$$

Interpretação: Isso é uma penalização adicional parametrizada pelo usuário. Antes de usar como resultado físico definitivo, é necessário confirmar se o laptime do Batch já contempla o comportamento das trocas para evitar dupla contagem.

11.3 Validação do modelo

A GUI oferece comparação de Top Speed $\times$ FDR e Aceleração Máxima $\times$ FDR entre o modelo analítico e as séries correspondentes do Batch Run. A finalidade é permitir calibração do modelo contra os resultados do OptimumLap antes de usar o modelo analítico como ferramenta de decisão.

12. Interface de terminal

A classe InterfacePlotterPowertrain mantém o fluxo histórico por terminal. Ela carrega arquivos independentemente, monta o objeto Carro, armazena resultados em um dicionário e exporta dados.

Método	Função
<code>_float / _int</code>	Leitura segura de parâmetros numéricos.
<code>_sim_nao</code>	Perguntas booleanas no terminal.
<code>_parametros_fisicos_efetivos</code>	Combina carro.csv, Batch e defaults.
<code>_montar_carro</code>	Cria Carro quando configuração + mapa existem.
<code>carregar_carro</code>	Lê configuração.
<code>carregar_rpm_torque</code>	Lê/valida mapa do motor.
<code>carregar_batch</code>	Lê Batch Run.
<code>mostrar_resumo_carro</code>	Imprime parâmetros principais.
<code>_salvar_resultado</code>	Coloca estudo na memória.
<code>_exportar_dados_genericos</code>	Serializa diferentes formatos para CSV.
<code>executar_grafico / executar_batch</code>	Despacham estudos pelo menu.

13. Interface gráfica

PlotterPowertrainGUI usa Tkinter/ttk para controles e incorpora Matplotlib com FigureCanvasTkAgg. A aplicação mantém três estados de entrada: configuração do carro, curva RPM $\times$ Torque e Batch Run.

Área	Comportamento
Cabeçalho	Mostra a logo e o nome da aplicação; fornece botões de carregamento dos três arquivos.
Dados carregados	Apresenta resumo do veículo e estado do Batch.
Estudo/gráfico	ComboBox seleciona o estudo e os parâmetros são atualizados dinamicamente.
Gráfico	Mostra a figura embutida e barra de navegação Matplotlib.
Dados gerados	Treeview apresenta os dados da saída.
Resultados em memória	Lista os estudos previamente gerados.
Exportação	CSV do resultado atual/selecionado/todos e imagem do gráfico.

13.1 Fluxo de executar_estudo()

1. Obtém o estudo selecionado e verifica se carro.csv/RPM $\times$ Torque/Batch são necessários.
2. Lê resolução, FDR e demais parâmetros da GUI.
3. Prepara uma nova Figure.
4. Despacha para a rotina correspondente.
5. Armazena o retorno em app.resultados.
6. Atualiza tabela, lista de resultados e aba do gráfico.
7. Informa o estado no rodapé da aplicação.

13.2 Normalização para tabela

`_normalizar_tabela` converte escalares, dicionários, listas [X,Y] e séries paralelas para o formato exigido pelo Treeview. A GUI mostra no máximo 5000 linhas visualmente por responsividade; a exportação pode continuar contendo todos os dados.

14. Mapa completo de funções e métodos

Função	Linhas	Descrição curta
<code>setMatplotlibParameters()</code>	118–200	
<code>interp_1d()</code>	204–218	Realiza interpolação linear 1D (substituindo <code>np.interp</code>).
<code>linspace_pure()</code>	220–228	Gera uma lista de números igualmente espaçados dentro de um intervalo.
<code>plotterPrint()</code>	230–231	
<code>_detectar_delimitador_csv()</code>	233–246	Tenta identificar o separador do arquivo de dados.
<code>getRPMxTorquePoints()</code>	249–287	Lê uma curva de motor e devolve [[RPM, torque], ...].
<code>getGearRatios()</code>	290–319	Lê as relações de marcha exportadas/copiadas do OptimumLap.
<code>_normalizar_rotulo()</code>	322–327	Normaliza um rótulo para comparações tolerantes a maiúsculas/acentos.
<code>_primeiro_valor_numerico()</code>	330–337	Converte um campo em float, retornando None quando ele não é numérico.
<code>_extrair_series_numericas()</code>	340–349	Extrai todos os números presentes depois do primeiro termo da linha.
<code>_buscar_linha_por_rotulo()</code>	352–369	Busca uma série do Batch Run pelo primeiro campo da linha.
<code>getBatchVehicleParameters()</code>	372–406	Extrai parâmetros físicos constantes disponibilizados pelo OptimumLap.
<code>getBatchRun()</code>	409–471	Lê um Batch Run do OptimumLap procurando os dados pelo primeiro termo.
<code>plotBatchLaptimesFDRShift()</code>	474–512	Plota FDR x laptime considerando um tempo adicional por troca.
<code>plotBatchLaptimeFDR()</code>	515–547	Plota FDR x laptime de cada run do Batch Run.
<code>getCarroCSV()</code>	550–564	Lê o carro.csv no formato de duas colunas "Parâmetro;Valor".
<code>_caminho_ah_absoluto()</code>	567–577	Reconhece caminhos absolutos tanto no Windows quanto no sistema atual.
<code>getCarroConfig()</code>	580–883	Lê e interpreta o arquivo de configuração "carro.csv".
<code>iniciar_gui()</code>	3571–3574	

Classe Carro — linhas 886–1849

Método	Linhas	Descrição curta
<code>__init__()</code>	900–1005	Inicializa o modelo de powertrain e seus parâmetros físicos.
<code>_gerar_mapa_operacional()</code>	1007–1033	Recorta/interpola o mapa até o limite de RPM efetivamente simulável.
<code>mapa_cobre_limite_rev()</code>	1036–1038	Indica se o limitador declarado excede o mapa fornecido.
<code>potencia_motor_hp()</code>	1040–1042	Retorna a potência do motor [HP] interpolada no mapa operacional.
<code>potencia_roda_hp_em_rpm()</code>	1044–1046	Retorna a potência disponível na roda após a eficiência do drivetrain.
<code>curvas_potencia()</code>	1048–1059	Retorna (velocidade, potência na roda, marcha) para cada marcha.
<code>curvas_torque()</code>	1061–1073	Retorna (velocidade, torque na roda, marcha)

		para cada marcha.
curvas_marcha()	1075–1094	Retorna curvas por marcha no formato (velocidade, força, índice).
forca_resistencia()	1096–1101	Calcula resistência aerodinâmica + resistência de rolamento [N].
downforce_total()	1103–1106	Calcula downforce total [N] usando $C_l * A$ e a densidade do ar.
_normal_traseira()	1108–1120	Carga normal no eixo motriz traseiro [N].
_limite_atrito()	1122–1124	Limite de força trativa no eixo motriz [N].
forca_trativa_disponivel()	1126–1159	Maior força trativa disponível [N] em uma velocidade.
forca_liquida()	1161–1163	Força líquida para aceleração [N] = tração - resistências.
velocidade_maxima_marcha()	1165–1169	Velocidade máxima cinemática da marcha no limite de RPM [km/h].
calcular_pontos_troca()	1171–1242	Encontra os pontos de troca pelo cruzamento das forças de marchas consecutivas.
calcular_top_speed()	1244–1272	Calcula top speed por equilíbrio entre tração disponível e resistências.
calcular_aceleracao_maxima()	1274–1296	Calcula a capacidade máxima de aceleração longitudinal [m/s ²].
simular_aceleracao_fdr()	1298–1386	Simula longitudinalmente uma aceleração 1D com trocas de marcha.
avaliar_fdrs()	1388–1404	Avalia vários FDRs por métricas diretamente ligadas ao desempenho longitudinal.
drawRPMxTorquexPower()	1406–1445	
gerar_pot_vroda()	1447–1463	Plota potência na roda por marcha. Retorna as curvas usadas.
gerar_troda_vroda()	1465–1484	Plota torque na roda por marcha, já considerando eficiência.
gerar_froda_vroda()	1486–1503	Plota força trativa por marcha, sem confundir força disponível com força líquida.
gerar_curva_unica_froda_vroda_v1()	1505–1531	Gera a envoltória de força disponível na roda para um FDR.
gerar_curva_unica_froda_vroda_v2()	1533–1580	Gera a envoltória de máxima força trativa disponível.
calcular_perda_area_potencia()	1582–1642	Métrica secundária de aproveitamento: área real / potência ideal.
calcular_perda_area_potencia_range()	1644–1649	Calcula a métrica secundária de perda de área para vários FDRs.
gerar_pot_vroda_unificada()	1651–1748	Gera e plota a curva unificada de máxima potência na roda vs velocidade.
comparar_pot_vroda_fdrs()	1750–1849	Calcula e plota no mesmo gráfico as curvas unificadas de potência na roda

Classe InterfacePlotterPowertrain — linhas 1857–2511

Método	Linhas	Descrição curta
init()	1869–1880	
_float()	1886–1898	
_int()	1901–1913	
_sim_nao()	1916–1926	
_parametros_fisicos_efetivos()	1928–1956	Combina parâmetros do carro.csv com valores do Batch Run.
_montar_carro()	1958–1978	Cria o objeto Carro quando configuração + curva do motor estão disponíveis.
carregar_carro()	1980–2001	Carrega somente as configurações do veículo.
carregar_rpm_torque()	2003–2025	Seleciona o arquivo externo da curva RPM x Torque e remonta o carro.
carregar_batch()	2027–2044	Seleciona e interpreta um CSV exportado do Batch Run do OptimumLap.
mostrar_resumo_carro()	2046–2070	
_salvar_resultado()	2072–2074	
_escrever_linhas_csv()	2080–2084	

exportar_resultado()	2086–2124	
_exportar_dados_genericos()	2126–2181	Converte os principais formatos retornados pelo código em CSV.
exportar_dados_base()	2183–2195	
exportar_todos()	2197–2208	
executar_grafico()	2213–2314	
executar_batch()	2319–2344	Executa um dos estudos do Batch Run já carregado.
executar_estudo_avancado_terminal()	2349–2397	Executa os estudos longitudinais/validação também pela interface terminal.
limpar_tela()	2400–2402	
menu()	2404–2442	
listar_resultados()	2444–2457	
executar()	2459–2511	

Classe PlotterPowertrainGUI — linhas 2524–3568

Método	Linhas	Descrição curta
init()	2531–2554	
_criar_estilo()	2559–2568	
_criar_interface()	2570–2738	
_criar_parametros()	2740–2767	Cria os campos de parâmetros que a GUI mostra/oculta por estudo.
_atualizar_parametros()	2769–2810	
_parametros_fisicos_efetivos()	2815–2839	Combina carro.csv + Batch Run sem sobrescrever valores explicitamente cadastrados.
_reconstruir_carro()	2841–2873	Monta o objeto Carro usando exatamente o mesmo pipeline do terminal.
carregar_carro()	2875–2894	Abre o carro.csv e carrega somente as configurações do veículo.
carregar_rpm_torque()	2896–2915	Abre o CSV externo da curva RPM x Torque.
carregar_batch()	2917–2942	Abre um CSV de Batch Run e o mantém disponível para os estudos.
_mostrar_dados_carregados()	2944–3012	Atualiza o painel textual com tudo que está atualmente carregado.
_curvas_marcha()	3017–3024	
_plot_engine()	3026–3042	
_plot_gear_curves()	3044–3054	
_plot_unified_force()	3056–3077	
_plot_perda_fdr()	3079–3096	
_plot_analitico_fdr()	3098–3117	
_plot_batch_validation()	3119–3145	
_plot_batch_laptime()	3147–3183	Desenha os estudos de Batch Run na figura embutida da GUI.
executar_estudo()	3188–3367	
_preparar_figura()	3369–3373	
_finalizar_figura()	3375–3392	
limpar_grafico()	3394–3407	
_normalizar_tabela()	3412–3448	Converte resultados comuns em colunas/linhas para o Treeview.
_atualizar_tabela()	3450–3469	
_atualizar_resultados()	3471–3477	
_selecionar_resultado()	3479–3487	
exportar_atual()	3492–3496	
exportar_selecionado()	3498–3503	

_exportar_nome()	3505–3518	
exportar_todos()	3520–3535	
salvar_grafico()	3537–3552	
_atualizar_status()	3554–3568	

15. Guia de manutenção

15.1 Onde alterar os parâmetros físicos

Primeira escolha: carro.csv. Para parâmetros que variam com o estudo e possuem um valor explícito no carro.csv, ele tem prioridade. Para parâmetros ausentes, _parametros_fisicos_efetivos pode preencher a partir do Batch ou de defaults.

Boa prática: Evite criar novos números mágicos dentro de métodos de plotagem. Se um valor for físico ou de configuração, prefira adicioná-lo ao carro.csv, ao mapeamento de parâmetros do Batch e ao construtor de Carro.

15.2 Onde alterar a física

Necessidade	Local recomendado
Potência do motor	Carro._init_ e _gerar_mapa_operacional
Velocidade por RPM	curvas_potencia / curvas_torque / curvas_marcha
Eficiência de transmissão	construtor + curvas_*
Arrasto / rolamento	forca_resistencia
Downforce	downforce_total
Transferência de carga	_normal_traseira
Limite de pneu	_limite_atrito
Escolha da marcha	forca_trativa_disponivel + calcular_pontos_troca
Top speed	calcular_top_speed
Aceleração temporal	simular_aceleracao_fdr
Comparação de FDR	avaliar_fdrs / GUI_plot_analitico_fdr

15.3 Onde alterar o Batch Run

Acrescente aliases ao mapping ou passe um novo rótulo para _buscar_linha_por_rotulo. Evite depender da ordem das linhas exportadas pelo OptimumLap. O primeiro campo da linha é a chave conceitual do dado.

15.4 Onde alterar a GUI

A lista de estudos está em PlotterPowertrainGUI._criar_interface. Os parâmetros dinâmicos são criados em _criar_parametros e ajustados em _atualizar_parametros. A lógica de cada estudo fica em executar_estudo, enquanto os gráficos reutilizáveis estão nos helpers _plot_*.

16. Limitações e cuidados conhecidos

Tema	Situação atual / cuidado
Mapa do motor	Só representa a faixa de RPM fornecida; não extrapolar torque fora dela sem uma hipótese explícita.
Largada	Não há embreagem nem modelo de lançamento; 0→alvo é

	aproximado.
Aderência	Modelo de eixo motriz com μ constante; não substitui um modelo de pneu completo.
Aerodinâmica	C_d/C_l são tratados como constantes; distribuição de aero é paramétrica.
Transferência de carga	Existe apenas uma aproximação 1D de pitch longitudinal.
Trocas	Tempo de troca é uma penalização fixa durante a simulação; não há dinâmica de torque/embreagem.
Curva unificada	É uma envoltória de capacidade, não uma simulação temporal por si só.
Perda de área	É métrica auxiliar; pode ter pouca sensibilidade ao FDR e não representa diretamente laptime.
Batch laptime + shift	Verificar se a penalização já está representada no modelo do OptimumLap antes de adicionar outra.
Convergência	Resultados dependem de resolução res/dt ; testes de sensibilidade são recomendados.

17. Procedimento recomendado de uso

8. Coloque carro.csv e arquivos auxiliares em uma estrutura de pastas estável; prefira caminhos relativos no carro.csv.
9. Abra o programa e carregue o carro.csv.
10. Carregue a curva RPM×Torque que será usada na análise.
11. Carregue o Batch Run apenas quando for necessário validar/comparar com o OptimumLap.
12. Confira o painel de dados atualmente carregados antes de gerar qualquer estudo.
13. Gere primeiro RPM×Torque×Potência, Potência/Torque/Força por marcha e pontos de troca para verificar coerência básica.
14. Use a simulação longitudinal para comparar FDRs em termos de tempo até a velocidade alvo e top speed.
15. Use as rotinas de validação Batch para verificar divergências do modelo analítico.
16. Só depois exporte os resultados finais para documentação/decisão.

18. Checklist de manutenção antes de modificar o código

- Preservar a separação entre leitura de dados, cálculo físico e plotagem.
- Não introduzir índices fixos para séries de Batch Run.
- Manter todas as unidades explícitas e documentadas.
- Ao alterar uma equação usada por mais de um estudo, centralizar a mudança nas funções da classe Carro.
- Testar pelo menos: leitura dos três arquivos, criação do Carro, pontos de troca, top speed, uma simulação e uma validação Batch.
- Comparar resultados antes/depois de mudanças físicas usando o mesmo mapa, veículo, FDR, res e dt .
- Registrar novas hipóteses físicas no carro.csv e neste manual.

19. Dados de teste e rastreabilidade

Para reproduzir uma análise, registre pelo menos: nome do carro, caminho/identificação do carro.csv, arquivo RPM×Torque, arquivo de Gear Ratios, Batch Run utilizado (quando houver), FDR(s), resolução res, dt, velocidade alvo, tempo de troca e se a aderência foi considerada.

Registro mínimo:

carro = ...

RPM×Torque = ...

GearRatios = ...

Batch = ...

FDRs = [...]

res = ... km/h

dt = ... s

velocidade_alvo = ... km/h

shift_time = ... s

considerar_atrito = True/False

Apêndice A — Equações em uma página

Grandeza	Equação	Unidade principal
Potência do motor	$P[\text{kW}] = \text{RPM} \cdot T / 9549.2966$	kW
Potência HP	$P[\text{HP}] = P[\text{kW}] \cdot 1.34102$	HP
FDR	$\text{FDR} = i_{\text{primária}} \cdot i_{\text{secundária}}$	-
Velocidade	$v = (\text{RPM}/60) \cdot (2\pi r / i_{\text{total}}) \cdot 3.6$	km/h
Torque na roda	$T_{\text{roda}} = T_{\text{motor}} \cdot i_{\text{total}} \cdot \eta$	N·m
Força de tração	$F = T_{\text{roda}} / r$	N
Arrasto	$F_{\text{drag}} = 0.5 \cdot \rho \cdot C_d \cdot A \cdot v^2$	N
Downforce	$F_{\text{down}} = 0.5 \cdot \rho \cdot C_l \cdot A \cdot v^2$	N
Rolamento	$F_{\text{roll}} = C_{rr} \cdot m \cdot g$	N
Força líquida	$F_{\text{net}} = F_{\text{trativa}} - F_{\text{res}}$	N
Aceleração	$a = F_{\text{net}} / m$	m/s ²
Limite de pneu	$F_{\text{pneu,max}} = \mu_x \cdot N_{\text{rear}}$	N
Top speed	$F_{\text{net}}(V^*) = 0$	km/h
Perda de área	$(A_{\text{ideal}} - A_{\text{real}}) / A_{\text{ideal}}$	adimensional

Apêndice B — Pontos de entrada no código por linha

Área	Linhas
Configuração/identidade visual	14–23
Visão geral/formato dos arquivos	26–113
Utilitários e parsers	118–886
Classe Carro	886–1850 aproximadamente
Interface de terminal	1857–2523
Interface GUI	2524–3574
Ponto de entrada	3571–3582

Apêndice C — Convenções de manutenção

Nomes de atributos e métodos preservam a nomenclatura existente para facilitar comparação com versões anteriores. Ao criar novos módulos, recomenda-se manter docstrings com: objetivo, entradas,

unidades, saídas, hipótese e limitações. Para alterações nas equações, registre a equação antiga e a nova em um changelog do projeto.